

Terrain Engine

A Real-Time OpenGL Terrain, Skybox, and Water Scene

Melvyn Qu

May 30, 2026

1 Overview

Terrain Engine is a real-time 3D scene written in C++ with the OpenGL 3.3 core profile. The viewer spawns above an island and flies freely through a world made of three cooperating subsystems: a height-map terrain with detail multi-texturing, a manually mapped skybox, and an animated, reflective water plane. The application is a single flat render loop (no scene graph) so that the order and the GL state of each pass are fully explicit.

2 Implementation Details

2.1 Terrain

The terrain mesh is generated at start-up from `heightmap.bmp`. Each grayscale pixel (i, j) becomes a vertex

$$x = (i - \frac{W}{2}) s_{xz}, \quad y = \frac{h(i,j)}{255} s_y + y_0, \quad z = (j - \frac{H}{2}) s_{xz},$$

and adjacent samples are stitched into two triangles per grid cell using an index buffer. The color texture is sampled with the natural grid UVs, while a second *detail* texture is tiled (`uDetailTiling`) to add high-frequency surface variation. The two textures are combined in the fragment shader with the *signed-add* rule, the modern GLSL equivalent of the fixed-function `GL_ADD_SIGNED` mode recommended in the brief:

```
vec3 combined = clamp(baseColor + detailColor - 0.5, 0.0, 1.0);
```

Subtracting 0.5 keeps the average brightness unchanged and avoids the colour clamping artefacts that `GL_ADD` or `GL_MODULATE` would introduce.

2.2 Skybox

The sky is a single high-resolution `GL_TEXTURE_CUBE_MAP`. It is loaded from one horizontal-cross PNG (4×3 cells); the six faces are sliced straight out of the decoded image using `GL_UNPACK_ROW_LENGTH`, so no external image tooling is needed. The cube is rendered by sampling the cubemap with the per-vertex direction `mat3(model) · aPos`. A pleasant consequence is that the water *reflection* comes for free: the reflection pass already scales the cube by $(1, -1, 1)$, which flips the sampling direction in Y and produces a mirrored sky with no extra code. Wrapping is `GL_CLAMP_TO_EDGE` on all three axes to avoid edge seams. (An earlier version mapped five separate 2D textures onto the cube faces with manual per-face UV rotations; the cubemap replaced that, both sharpening the sky and removing the orientation bookkeeping.)

2.3 Water

The bottom plane samples a 128×128 wave texture with `GL_REPEAT` and a tiling factor well above 1.0 so the low-resolution image is not stretched. Animation is achieved by scrolling the UVs every frame by an accumulated `uWaveShift` (advanced by $\text{speed} \times \Delta t$ so the motion is frame-rate independent):

```
vec2 uv = vec2(TexCoords.x * uWaterTiling + uWaveShift,
               TexCoords.y * uWaterTiling + 0.5 * uWaveShift);
```

The surface is alpha-blended over the scene and tinted slightly toward a sea-blue. A distance-based `smoothstep` fade makes the water more transparent near the camera (revealing the reflection) and opaque toward the horizon, which hides the far edge of the plane.

2.4 Reflections

Two reflections are rendered. The *sky* reflection re-draws the skybox cube scaled by $(1, -1, 1)$ below the water line with depth writes disabled. The *terrain* reflection multiplies the terrain model matrix by a mirror transform about the water plane, $T(0, 2y_w, 0) \cdot S(1, -1, 1)$. Because the negative scale reverses triangle winding, the front-face convention is temporarily switched to `GL_CW` for that draw and restored afterwards. Each terrain pass is bounded by a user clip plane fed through `gl_ClipDistance[0]`: the reflection keeps only geometry below the water surface, and the real pass keeps only geometry above it.

2.5 Camera and asset paths

Movement uses a standard Euler fly-camera (`camera.h`) driven by WASD plus Space/Shift, mouse-look, and scroll-to-zoom. As required by the brief, motion is *smooth*: instead of teleporting the position each frame, the camera keeps a velocity vector that exponentially approaches the target speed while a key is held (gradual acceleration to a cruising speed) and decays back to zero when released (gradual deceleration). The exponential approach $v += (v_{\text{target}} - v)(1 - e^{-r \Delta t})$ makes the feel frame-rate independent. All asset and shader paths are resolved from a compile-time `PROJECT_ROOT` string injected by CMake, so the executable loads its resources correctly regardless of the current working directory.

3 Challenges & Solutions

Machine-specific hard-coded paths

The project originally embedded machine-specific absolute paths (rooted under a `computer_graphics/home` directory) for every shader and texture, and one of them pointed at a directory that no longer existed, so asset loading failed at runtime. All paths were rewritten to be relative to the CMake-provided `PROJECT_ROOT`, and the build configuration was repointed at the in-repository `external/` dependencies.

Broken / missing build dependencies

The build referenced a GLFW `.dylib` that was no longer present (only the static `libglfw3.a` remained) and a FreeType library that was not installed. The fix was to link the static GLFW archive together with the required macOS frameworks (`Cocoa`, `IOKit`, `CoreVideo`, `OpenGL`), and to install the remaining toolchain (CMake, and FreeType while the text overlay still existed).

Skybox seams and orientation

With the default clamp mode the cube edges showed visible seams, and several face images were

rotated relative to the cube. Switching to `GL_CLAMP_TO_EDGE` removed the seams, and per-face UV-rotation uniforms aligned every image.

Reflection winding

Mirroring the terrain with a negative scale inverted the triangle winding, which caused the reflected terrain to be culled. Temporarily setting `glFrontFace(GL_CW)` around the mirrored draw resolved it.

HUD text orientation (documented, then removed)

An on-screen FreeType HUD displayed the camera position, but the glyphs rendered flipped because FreeType stores bitmaps top-row-first while the texture sampling assumed the opposite. After several iterations on the vertical flip, the HUD was intentionally removed from the final build to keep the scene clean and to drop the FreeType dependency entirely. This is a deliberate scope decision, not an unresolved bug.

Note on orientation. Because cubemap faces follow a fixed convention, swapping in a new sky is now a matter of replacing one cross PNG; if a particular set is authored with a different face order, only the cross-cell mapping in `loadCubemapCross` needs adjusting.

4 Environment Setup

Operating system	macOS (Darwin 25.4, Apple Silicon)
Compiler / standard	Apple Clang, C++14
Build system	CMake (≥ 3.5)
Windowing / input	GLFW 3.4 (static <code>libglfw3.a</code>)
GL loader	GLAD (OpenGL 3.3 core)
Math library	GLM
Image loading	<code>stb_image</code>

GLAD, GLFW, GLM, and `stb_image` are vendored under `external/`; only CMake must be installed to build. The application is built with `cmake -B build && cmake --build build` and runs as `./terrainEngine`.

5 Bonus & References

Features beyond the basic requirements

- **Dual reflection** of both the sky and the terrain in the water, each clipped to the correct half-space with `gl_ClipDistance`.
- **Distance-based water transparency** using a `smoothstep` fade, so the water reveals the reflection up close and hides the plane edge at range.
- **Frame-rate-independent** wave animation via Δt accumulation.
- **Portable asset resolution** through a compile-time `PROJECT_ROOT` define, allowing the binary to run from any directory.
- **Self-contained build:** all third-party dependencies vendored in-repo.

- **4× MSAA** for anti-aliased geometry edges.
- **High-resolution cubemap sky** sliced in-code from a single cross image.

Possible future improvements

- Per-vertex or normal-mapped **terrain lighting** (currently unlit albedo).
- **GPU-displaced** water with real normals for specular sun glint, instead of a scrolling texture.
- **Fresnel-weighted** reflection blending for a more physically convincing water surface.
- **Frustum culling** / **LOD** on the terrain grid for larger maps.
- Higher-resolution or procedurally generated sky.

References

1. Joey de Vries, *LearnOpenGL* — <https://learnopengl.com/> (camera, shaders, text-rendering, skybox tutorials).
2. GLFW 3.4 documentation — <https://www.glfw.org/docs/latest/>.
3. OpenGL Mathematics (GLM) — <https://github.com/g-truc/glm>.
4. Sean Barrett, **stb_image** — <https://github.com/nothings/stb>.
5. Course design brief, `doc/TerrainDoc.md` (skybox / water / terrain spec).
6. “Cloudy Skyboxes” by Screaming Brain Studios, CC0 / public domain — <https://opengameart.org/content/cloudy-skyboxes-0> (skybox cubemap art).